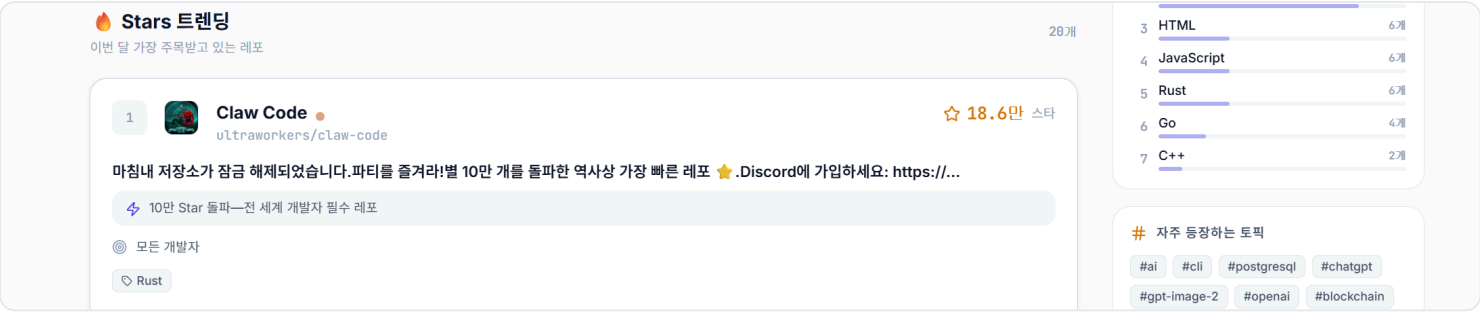
PROJECT 01

QUOKIT — LangGraph 기술 리서치 Agent & 큐레이션

2026.01 ~ 현재 · 개인 프로젝트 · AI Agent / Full-stack

LangGraph LangChain Python Next.js 16 TypeScript Supabase Upstash Redis Tailwind CSS

GitHub Trending 오픈소스를 실시간 수집하고, LangGraph 기반 Multi-Agent(리서치–원문검증–리포트 생성) 파이프라인을 통해 기술 스택 분석과 신뢰성 있는 심층 요약을 제공하는 AI 큐레이션 플랫폼입니다.



Problem

영문 README의 단순 직역 요약만으로는 오픈소스의 실제 실무 아키텍처와 기술 스택의 신뢰도를 검증하기 어려움

단일 LLM 호출 방식 사용 시 라이브러리 기능이나 버전을 잘못 단정하는 **환각(Hallucination) 및 근거 누락** 발생

대량의 오픈소스 저장소를 매일 반복 분석할 경우 급증하는 **LLM 토큰 비용 및 응답 지연** 발생

Solution

LangGraph StateGraph를 도입하여 조사(Research)와 검증(Verification) 노드를 분리하고 **자가 교정(Self-Correction)** 루프 구축

LangChain Structured Output & Pydantic 스키마를 적용해 핵심 기능, 기술 스택, 제약사항을 강타입 데이터로 정규화

커밋 SHA 기반 **Upstash Redis 1차 캐싱**과 **Supabase(PostgreSQL)** 영속화로 중복 LLM 호출 90% 이상 차단

System Architecture

트렌딩 수집 (Cheerio + GitHub API) → Research Agent (원문 및 코드 심층 탐색) → Verification Agent (근거 교차 검증 & 피드백) → Report Agent (Pydantic 구조화 리포트) → Redis & DB (Upstash 캐시 + Supabase) → Next.js UI (큐레이션 대시보드 서빙)

Key Features

LangGraph Multi-Agent 파이프라인 — Research–Verification–Report 노드 분업 기반의 신뢰성 있는 리서치

Pydantic 스키마 Structured Output — 핵심 아키텍처, 기능, 제약사항을 강타입 스키마로 검증하여 DB 무결성 확보

GitHub Trending Stars/Forks 큐레이션 — 글로벌 화제성과 실용성 지표 기반의 일간·주간·월간 실시간 랭킹 제공

조건부 엣지 기반 자가 교정(Self-Correction) — 근거 일치성 미달 시 State 카운트에 따른 자동 재탐색 분기 수행

커밋 SHA 기반 Redis 다층 캐시 — Upstash Redis 1차 캐시를 통해 중복 LLM 호출 90% 절감 및 API 한도 방어

언어별·토픽별 다차원 필터링 — Python, TypeScript, Rust 등 언어별 랭킹 및 #ai, #cli 등 연관 토픽 검색 지원

개발 중 겪은 문제

문제

단일 프롬프트 일괄 처리 시 환각 발생 — 방대한 README 요약 시 라이브러리 기능이나 버전을 왜곡하는 현상 발생

비결정적 LLM 출력으로 인한 DB 적재 오류 — 자유 텍스트 포맷 시 JSON 파싱 실패 및 필수 필드 결측 빈발

트렌딩 갱신 시 반복 호출로 인한 비용 급증 — 잦은 랭킹 변동 시 동일 레포에 대한 중복 분석으로 토큰 과금 발생

해결

LangGraph StateGraph로 리서치 노드와 검증 노드를 분리하고, 원문 인용 근거가 부족하면 재탐색하는 피드백 루프 구축

LangChain Structured Output & Pydantic을 결합하여 엄격한 타입 스키마로 검증 후 안전하게 영속화

최신 **커밋 SHA 해시를 캐시 키**로 사용하는 Upstash Redis 레이어를 구축하여 코드 변경이 없을 시 캐시 즉시 반환

사용자 경험 흐름

트렌딩 랭킹 확인 → 언어/토픽 필터 선택 → Agent 검증 리포트 열람 → 기술 스택·근거 파악 → GitHub 원본 저장소 방문

사용자는 Stars/Forks 지표로 최신 인기 레포를 확인하고, LangGraph Multi-Agent가 교차 검증한 고신뢰도 한글 리서치 리포트를 통해 오픈소스의 핵심 기술 가치를 빠르게 파악할 수 있습니다.

Result & Learning

- ✓ LangGraph를 활용한 상태 관리 및 Multi-Agent 오케스트레이션 아키텍처를 프로덕션 레벨로 설계·구현했습니다
- ✓ LLM 환각 문제를 프롬프트에만 의존하지 않고, 검증 노드와 조건부 엣지(자가 교정 루프)를 결합한 신뢰성 엔지니어링을 체득했습니다
- ✓ Python LangGraph 에이전트 백엔드와 Next.js 16 풀스택 웹 서비스를 결합한 고도화된 AI 제품화 역량을 확보했습니다

프로젝트 저장소 보기 →

PROJECT 02

Daily Emotion Log

2025.12 · 5인 팀 · PM / Full-stack

HTML/CSS/JS Flask MariaDB Gemini API Cloudtype

Gemini API를 활용해 감정 기록을 분석하고, 날짜별 변화 흐름을 시각화하는 서비스를 설계·구현했습니다.



Problem

기존 일기 앱은 기록만 가능하고, 감정 패턴을 분석하는 기능이 없었습니다
사용자가 자신의 감정 변화를 스스로 인식하기 어려운 구조였습니다
기록된 감정에 대해 객관적 피드백을 받을 방법이 없었습니다

Solution

8가지 감정 선택과 메모 작성 후 **Gemini API**로 맞춤 피드백을 생성하도록 구성했습니다
날짜별 감정 데이터를 **차트와 타임라인**으로 시각화해 변화를 확인할 수 있도록 설계했습니다
주간·월간 **트렌드 리포트**를 자동 생성해 패턴을 요약했습니다

System Architecture

감정 입력 (8종 감정 선택 + 메모) → 프론트엔드 (입력 검증 및 UI) → Flask 서버 (프롬프트 분기 처리) → Gemini API (감정별 피드백 생성) → MariaDB (기록 + 분석 저장) → 차트 출력 (트렌드 시각화)

Key Features

감정 선택 UI — 8가지 감정을 직관적으로 선택하고 메모를 기록하는 화면을 구현했습니다

AI 피드백 — Gemini API로 감정 유형에 맞는 맞춤 분석을 생성하도록 연동했습니다

프롬프트 분리 — 감정마다 다른 프롬프트 템플릿을 적용해 응답 품질을 통일했습니다

감정 차트 — 날짜별 감정 변화를 라인 차트로 시각화하는 컴포넌트를 구현했습니다

트렌드 리포트 — 주간·월간 감정 패턴을 자동으로 요약하는 리포트를 설계했습니다

조회 최적화 — 날짜 범위 인덱스를 적용해 차트 로딩 속도를 개선했습니다

개발 중 겪은 문제

문제

8개 감정 버튼을 만들었는데 **어떤 걸 선택했는지 구분이 안 됨**
처음에는 색상만 바꿔서 사용자 테스트에서 "모르겠다" 피드백
Gemini API 응답 1~3초 동안 **로딩 피드백이 없어서** 답답한 UX
fetch 실패 시 에러 처리 없이 화면이 멈춰버리는 문제

해결

스케일 애니메이션 + 배경색 강조 + 체크 아이콘으로 선택 상태 명확화
스켈레톤 UI + 로딩 스피너로 대기 상태를 시각적으로 표시
응답 수신 시 **fade-in 전환**으로 자연스러운 전환 구현
try-catch + **재시도 버튼**으로 에러 상황 대응

사용자 경험 흐름

감정 선택 → 메모 작성 → AI 피드백 확인 → 날짜별 변화 확인 → 주간/월간 패턴 확인

사용자가 감정을 선택하고 메모를 남기면, AI가 즉시 피드백을 생성합니다. 이후 차트에서 날짜별 감정 추이를 확인하고, 주간·월간 리포트로 패턴을 파악할 수 있도록 구성했습니다.

Result & Learning

- ✓ 프론트엔드부터 Flask, Gemini API, MariaDB까지 전체 파이프라인을 처음부터 직접 설계하고 구현했습니다
- ✓ AI 프롬프트를 유형별로 분리하고 JSON으로 파싱하는 체계를 실전에서 구축했습니다
- ✓ PM을 맡아 스탠드업 미팅 도입, 일정 관리, 역할 분배를 병행하며 팀 리딩을 경험했습니다

프로젝트 저장소 보기 →

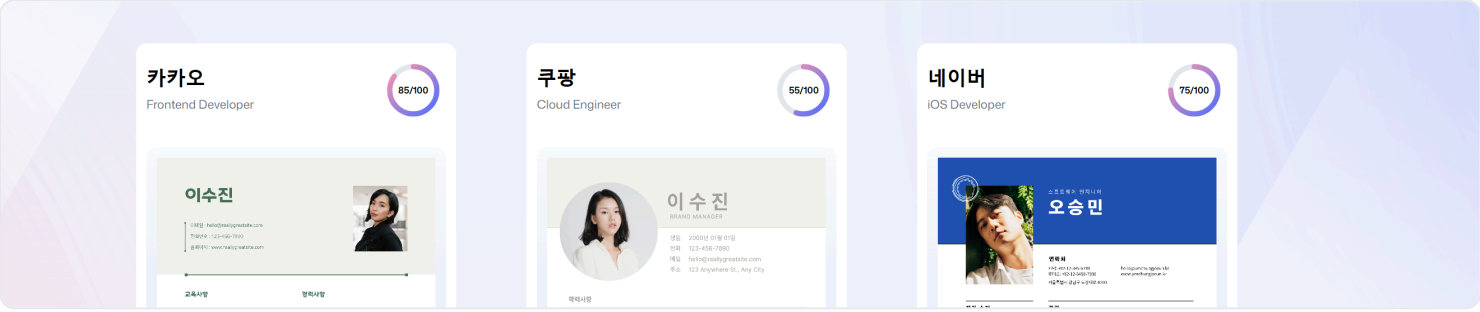
PROJECT 03

Ascend — 이력서 코칭 웹

2025.10–2025.12 · 개인 프로젝트

React TypeScript Vite Zustand React Query Puter.js Tailwind

이력서 PDF를 올리면 AI가 섹션별로 점수를 매기고, ATS 키워드 분석까지 제공하는 코칭 서비스를 설계·구현했습니다.



Problem

이력서를 여러 번 고쳐도 어디가 약한 건지 스스로 판단하기 어려웠습니다
ATS 관점에서 핵심 키워드가 부족한지 확인할 방법이 없었습니다
어떤 문장을 어떻게 바꿔야 하는지 알려주는 도구도 없었습니다

Solution

PDF 업로드 후 **정규표현식**으로 섹션을 자동 분리하도록 구현했습니다
Puter.js AI가 섹션별 1~100점 채점과 문장 단위 수정 제안을 생성합니다
이력서 키워드를 시각화하고, 업계 기준 대비 **누락 키워드**를 자동 제안합니다

System Architecture

PDF 업로드 (텍스트 추출) → 섹션 분리 (정규표현식 파싱) → Puter.js AI (채점 + 개선안) → Zustand (UI 상태 관리) → React Query (AI 응답 캐싱) → 키워드 맵 (누락 키워드 제안)

Key Features

PDF 섹션 분리 — 정규표현식으로 이력서를 경력/학력/스킬로 자동 구분하도록 구현했습니다

AI 채점 — 각 섹션을 1~100점으로 평가하고 문장 단위 수정 제안을 생성합니다

키워드 분석 — 이력서에 쓰인 키워드를 시각화하고 빠진 항목을 제안합니다

상태 분리 — Zustand(UI 상태)와 React Query(AI 응답)로 역할을 분리했습니다

응답 캐싱 — 같은 이력서를 다시 분석하면 캐시에서 즉시 결과를 돌려줍니다

문구 개선 — "이 문장은 이렇게 바꾸세요" 수준의 구체적 수정안을 생성합니다

개발 중 겪은 문제

문제

AI 분석 결과·입력값·탭 상태가 **여러 컴포넌트에 흩어져** 있었음
props 깊이 3~4단계 → **데이터 변경 위치 추적 불가**
같은 이력서 재분석 시 AI 재호출 → **불필요한 API 비용** 발생

해결

UI 상태(탭, 모달)는 **Zustand**, 서버 데이터는 **React Query**로 분리
React Query 캐싱으로 **동일 이력서 재호출 제거**
상태 관리 위치 명확화 → **버그 탐색 시간 50% 감소**
"클라이언트/서버 상태 분리"를 체감한 계기

사용자 경험 흐름

PDF 업로드 → 섹션별 점수 확인 → 수정 제안 확인 → 키워드 맵 확인 → 누락 키워드 제안

이력서를 올리면 AI가 즉시 섹션별 점수와 수정 방향을 알려주고, 키워드 맵에서 업계 기준 대비 빠진 키워드까지 확인할 수 있도록 구성했습니다.

Result & Learning

- ✓ 클라이언트 상태와 서버 상태를 분리해서 관리하는 아키텍처를 직접 설계하고 적용했습니다
- ✓ PDF 텍스트 추출부터 AI 채점, 키워드 분석까지 전체 파이프라인을 구축했습니다
- ✓ 타입을 먼저 정의하고 개발하는 방식이 이후 프로젝트의 기본 원칙으로 자리잡았습니다

프로젝트 저장소 보기 →

PROJECT 04

Interview Simulator

2025.09–2025.11 · 팀 프로젝트 · Frontend 담당

React TypeScript WebRTC Recharts Node.js Express MongoDB

카메라로 표정을 분석하고 음성을 텍스트로 변환해서, 면접 연습 결과를 동기화된 타임라인으로 시각화하는 서비스를 구현했습니다.



Problem

면접에서는 답변 내용뿐 아니라 표정·시선도 평가에 영향을 줍니다
답변과 비언어적 요소를 종합 분석해주는 연습 도구가 없었습니다
영상·음성·감정 데이터가 각각 다른 시점에 생성되어 합치기 어려웠습니다

Solution

AI가 면접 질문을 생성하고, **STT**로 답변을 실시간 인식하도록 구현했습니다
WebRTC 카메라로 표정과 집중도를 수치화하여 분석했습니다
질문·답변·감정을 같은 시간축에 맞춰 **타임라인**으로 시각화했습니다

System Architecture

WebRTC 카메라·마이크 스트림 → **표정 분석** 감정·집중도 수치화 → **STT 엔진** 실시간 텍스트 변환 → **상태 머신** 단계별 관리·롤백 → **MongoDB** 세션 데이터 저장 → **타임라인** 동기화 시각화

Key Features

AI 질문 생성 — 직무·경험에 맞는 면접 질문을 실시간으로 생성하도록 연동했습니다

표정 분석 — 카메라 영상에서 표정을 읽어 감정·집중도를 수치로 변환했습니다

상태 머신 — WebRTC 비동기 흐름을 단계별로 관리하고 에러 시 롤백 처리를 구현했습니다

음성 인식 — STT로 답변을 바로 텍스트로 변환해 기록하도록 구현했습니다

동기화 타임라인 — 질문·답변·감정을 같은 시간축에 정렬해 시각화했습니다

세션 리포트 — 답변 품질, 감정 변화, 개선 포인트를 종합 요약하도록 구성했습니다

개발 중 겪은 문제

문제

표정 데이터는 0.5초마다, STT는 문장 끝날 때 → **시점 불일치**
타임라인에서 **표정과 답변의 매칭이 안 됨**
카메라/마이크 권한 거부 시 **앱이 멈춰버리는** 문제

해결

모든 이벤트에 **ms 단위 타임스탬프** 부여 후 시간순 정렬
getUserMedia를 try-catch로 감싸고 권한 없으면 안내 메시지 표시
연결→녕화→분석→완료를 **단계별 상태**로 관리 → 에러 시 롤백 가능

사용자 경험 흐름

카메라 연결 → 질문 받기 → 답변 + 분석 → 타임라인 확인 → 종합 리포트

카메라를 연결하면 AI가 질문하고, 답변하는 동안 표정과 음성을 동시에 분석합니다. 면접이 끝나면 타임라인과 종합 리포트로 결과를 확인할 수 있습니다.

Result & Learning

- ✓ WebRTC→표정분석→STT→시각화로 이어지는 실시간 파이프라인을 직접 구현했습니다
- ✓ 타임스탬프 기반으로 여러 비동기 데이터를 동기화하는 방법을 배웠습니다
- ✓ API 명세를 먼저 문서화하면 프론트-백 통합이 훨씬 수월하다는 걸 체감했습니다

프로젝트 저장소 보기 →

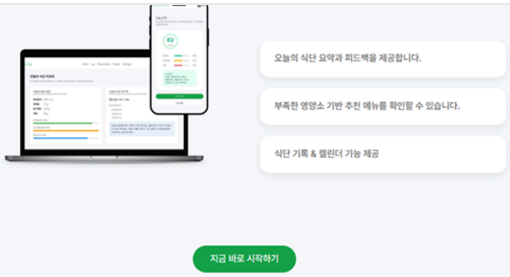
PROJECT 05

BalanceEat

2025.10–2025.11 · 4인 팀 · PO / Frontend

React TypeScript Gemini AI Nutrition API Figma

먹은 음식의 영양소를 자동으로 분석하고, 부족한 영양소에 맞춰 AI가 식단을 추천하는 서비스를 구현했습니다.



Problem

기존 식단 앱은 음식을 일일이 수동 입력해야 해서 사용자 부담이 컸습니다
영양을 분석해도 맞춤 추천까지 이어지지 않았습니다
어떤 영양소가 부족한지 사용자가 직접 판단하기 어려웠습니다

Solution

음식 이름을 입력하면 **Nutrition API**로 영양 정보를 자동 검색하도록 연동했습니다
3일치 섭취 데이터를 **Gemini AI**에 전달해 부족 영양소 기반 식단을 추천하도록 구성했습니다
영양소별 달성률을 **도넛 차트**로 실시간 표시하도록 설계했습니다

System Architecture

음식 검색 이름 + 섭취량 입력 → **React + TS** 타입 정의 + 검증 → **Nutrition API** 영양소 자동 조회 → **Local Storage** 브라우저 저장 → **Gemini AI** 부족 영양소 분석 → **차트 출력** 달성률 시각화

Key Features

영양 자동 검색 — 음식 이름만 입력하면 Nutrition API에서 정보를 불러오도록 연동했습니다

달성률 차트 — 칼로리·단백질·탄수화물·지방별 목표 달성률을 도넛 차트로 구현했습니다

타입 안전성 — TypeScript interface로 API 응답 구조를 사전 정의해 오류를 방지했습니다

AI 식단 추천 — 3일간 섭취를 분석해 부족 영양소 기반 식단을 제안하도록 설계했습니다

날짜별 기록 — 캘린더에서 날짜를 선택하면 식단과 영양 요약을 조회하도록 구성했습니다

오프라인 저장 — Local Storage를 활용해 서버 없이 데이터가 유지되도록 구현했습니다

개발 중 겪은 문제

문제

Nutrition API 응답 구조가 **음식마다 필드가 다름** (칼로리만 있고 단백질 없는 경우)
누락된 필드를 차트에 넘기면 **undefined 에러** 발생
영양소 4개 도넛 차트가 데이터 변경 시 **전체 리렌더링**

해결

기본값(0) 정규화 함수로 누락 필드 대응
TypeScript interface로 영양소 타입 사전 정의 → **컴파일 시점에 오류 감지**
React.memo + useMemo로 변경된 영양소만 다시 그리도록 최적화

사용자 경험 흐름

음식 검색 → 영양 자동 표시 → 달성률 차트 → AI 식단 추천 → 날짜별 기록 조회

음식 이름만 입력하면 영양 정보가 자동으로 표시되고, 하루 목표 대비 달성률을 차트로 확인할 수 있습니다. 3일 이상 기록하면 AI가 부족한 영양소를 분석해 맞춤 식단을 추천합니다.

Result & Learning

- ✓ "코딩알려주는누나" 강의를 마친 뒤 배운 것을 실전 서비스로 완성한 첫 팀 프로젝트입니다
- ✓ PO로서 Figma 와이어프레임을 먼저 그리고 팀원과 방향을 맞추는 과정을 주도했습니다
- ✓ TypeScript로 타입을 먼저 정의하는 습관이 형성되어 이후 프로젝트에도 기본으로 적용했습니다

프로젝트 저장소 보기 →

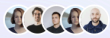
PROJECT 06

ResumeBuilder — 이력서 빌더

2025.10–2025.11 · 개인 프로젝트

React TypeScript Vite State Management

섹션별 입력 폼으로 이력서를 작성하면, 옆에서 실시간으로 미리볼 수 있는 이력서 빌더를 구현했습니다.



당신의 직업에 맞는 이력서를 만들어보세요
AI로 이력서를 생성합니다

나만의 전문적인 이력서를 생성하고 , 수정하고, 다운로드 해보세요 AI가
효과적인 도움을 제공합니다

Problem

빈 문서에서 이력서를 시작하면 뭘 어떻게 써야 할지 막막합니다
작성하면서 결과물을 바로 확인할 수 없어 수정을 반복하게 됩니다
항목별로 작성 방향을 안내해주는 도구가 없었습니다

Solution

경력·학력·스킬·프로젝트마다 **전용 입력 폼**과 작성 가이드를 제공했습니다
오른쪽 패널에서 **실시간 미리보기**를 보여주도록 설계했습니다
섹션을 자유롭게 추가·삭제할 수 있는 **동적 폼 관리**를 구현했습니다

System Architecture

사용자 입력 (섹션별 전용 폼) → 입력 검증 (유효성 + 가이드) → 단일 상태 트리 (상태 끌어올리기) → 메모이제이션 (변경분만 재렌더링) → 미리보기 패널 (실시간 반영)

Key Features

구조화된 폼 — 경력/학력/스킬/프로젝트마다 전용 입력 양식을 설계했습니다

실시간 미리보기 — 입력 즉시 오른쪽 패널에 이력서 형태로 반영됩니다

동적 섹션 관리 — 항목을 자유롭게 추가·삭제하고 유니크 키로 안정 렌더링합니다

작성 가이드 — 입력 필드에 포커스하면 작성 방향을 안내하는 UX를 구현했습니다

렌더링 최적화 — 메모이제이션으로 불필요한 리렌더링을 방지했습니다

상태 유지 — 페이지를 이동해도 입력 데이터가 유지되도록 구성했습니다

개발 중 겪은 문제

문제

글자 입력 시 **미리보기 전체가 리렌더링** → 입력 버벅거림
섹션 5개 이상일 때 **타이핑 체감 지연** 눈에 띄엄
동적 섹션 추가·삭제 시 **key 충돌**로 입력값 사라지는 버그

해결

React.memo로 미리보기 감싸고 **useMemo**로 섹션별 캐싱
동적 항목에 **uuid**로 고유 키 부여 → 삭제 후에도 입력값 유지
이 프로젝트의 상태 관리 한계가 Ascend에서 **Zustand 도입의 계기**

사용자 경험 흐름

섹션 선택 → 항목 입력 → 실시간 미리보기 → 섹션 추가/삭제 → 완성된 이력서

경력, 학력 등 섹션을 선택하고 내용을 입력하면 오른쪽에서 즉시 이력서 형태로 확인할 수 있습니다. 필요에 따라 섹션을 자유롭게 추가·삭제할 수 있도록 설계했습니다.

Result & Learning

- ✓ 여러 섹션의 폼 상태를 하나의 트리로 모아 관리하는 설계를 직접 해봤습니다
- ✓ 실시간 미리보기 성능을 유지하기 위한 메모이제이션 전략을 적용했습니다
- ✓ 상태 관리의 한계를 직접 느끼면서, 이후 Zustand 도입의 계기를 마련했습니다

프로젝트 저장소 보기 →

PROJECT 07

Velin — AI Interview Prep

2025.08–2025.09 · 개인 프로젝트 (Full-stack)

React TypeScript Node.js Vercel

직무와 경력을 입력하면 AI가 맞춤 면접 질문, 예상 답변, 체크리스트를 한 번에 생성하는 서비스를 설계·구현했습니다.



Problem

기술 면접 질문은 검색할 수 있지만, 내 직무·경력에 맞는 맞춤 질문을 찾기 어려웠습니다
질문에 어떤 방향으로 답해야 하는지 가이드해주는 도구가 없었습니다
면접 준비 과정을 체계적으로 기록하고 비교할 수 없었습니다

Solution

직무·포지션·경력을 입력하면 AI가 **맞춤 질문과 답변 방향**을 생성합니다
답변의 **논리적 근거**와 면접 전 **체크리스트**를 함께 생성하도록 설계했습니다
세션별 기록을 저장하고 이전 연습과 **비교 분석**할 수 있는 구조를 구성했습니다

System Architecture

직무 입력 (포지션·경력 선택) → React (검증 + 요청) → Node.js (조합 처리) → 프롬프트 엔진 (동적 템플릿 조합) → AI 응답 (JSON 파싱) → 세션 저장 (회차별 비교)

Key Features

동적 프롬프트 엔진 — 직무·포지션·경력을 조합해 맞춤 프롬프트를 설계했습니다

예상 답변 가이드 — 질문마다 답변 방향과 핵심 포인트를 함께 생성합니다

근거 생성 — 답변에 활용할 논리적 근거를 자동으로 생성합니다

면접 체크리스트 — 면접 전 점검 항목을 AI가 자동으로 구성합니다

세션 비교 — 회차별 기록을 저장하고 이전 연습과 비교하도록 구현했습니다

에러 처리 — AI 응답 지연 시 타임아웃과 재시도 로직을 구현했습니다

개발 중 겪은 문제

문제

AI 응답 **3~5초 걸리는 동안** 화면에 피드백이 없음
사용자가 "고장났나?" 하고 **새로고침을 누르는** 경우 빈발
같은 직무에도 **질문 품질이 들쭉날쭉** (하나의 프롬프트로 모든 직무 처리)

해결

"질문을 만들고 있습니다..." 로딩 메시지 + 진행 애니메이션 추가
5초 이상 시 **타임아웃 안내 + 재시도 버튼** 표시
직무·경력 수준별로 **프롬프트를 다르게 조합**하여 품질 편차 해소

사용자 경험 흐름

직무 선택 → 맞춤 질문 확인 → 답변 방향 확인 → 체크리스트 확인 → 이전 세션과 비교

직무와 경력을 선택하면 AI가 맞춤 질문, 예상 답변, 체크리스트를 한 번에 생성합니다. 회차별 기록이 저장되어 이전과 비교도 가능합니다.

Result & Learning

- ✓ 프론트엔드와 백엔드를 혼자 설계하고 구현한 첫 풀스택 프로젝트입니다
- ✓ 조건을 동적으로 조합하는 프롬프트 템플릿 엔진을 직접 구축했습니다
- ✓ Vercel CI/CD를 통해 프로덕션 배포와 환경 변수 관리를 경험했습니다

프로젝트 저장소 보기 →

데모 사이트 보기 →

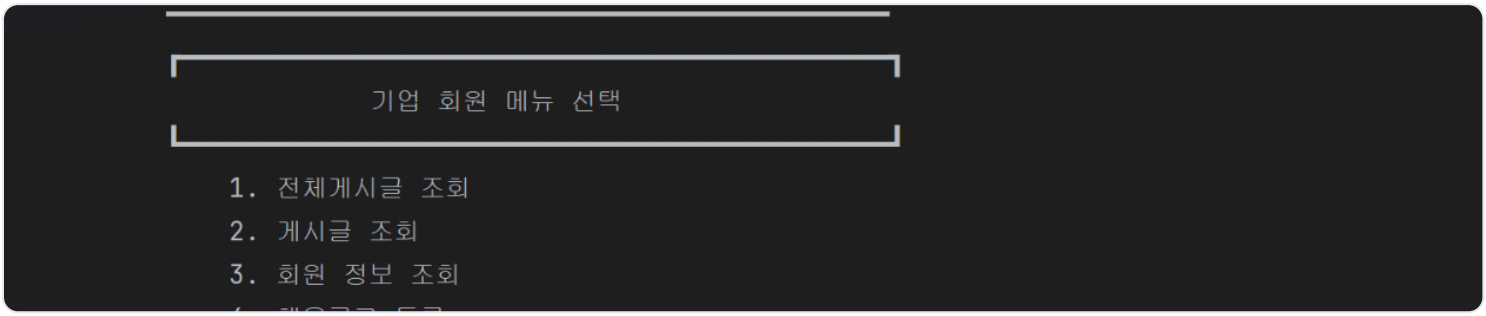
PROJECT 08

ALBAIT — 구인·구직 플랫폼

2025.04–2025.05 · 개인 프로젝트

Java Oracle DB JDBC SQL

Java와 Oracle DB로 회원·공고·지원 CRUD를 콘솔에서 구현하고, DAO/DTO 패턴으로 계층을 분리한 프로젝트입니다.



Problem

CRUD의 전체 사이클을 실무에 가까운 도메인으로 직접 구현해보고 싶었습니다
서비스 로직과 DB 접근 코드가 섞이면 유지보수가 어려워집니다
복잡한 집계 쿼리를 작성하고 최적화하는 경험이 필요했습니다

Solution

구인·구직이라는 **관계형 데이터가 풍부한** 도메인을 선택했습니다
DAO/DTO 패턴으로 서비스 로직과 데이터 접근 계층을 완전히 분리했습니다
GROUP BY·JOIN을 조합한 관리자 통계 기능을 구현했습니다

System Architecture

콘솔 UI (메뉴 기반 입력) → 서비스 계층 (비즈니스 로직) → DTO (데이터 전달 객체) → DAO (CRUD + 통계 쿼리) → JDBC (파라미터 바인딩) → Oracle DB (관계형 저장)

Key Features

회원 CRUD — 구직자/사업장 가입·로그인·수정·탈퇴를 전체 구현했습니다

지원 관리 — 지원 접수 후 상태(대기/합격/불합격) 업데이트를 설계했습니다

DAO/DTO 패턴 — 데이터 접근과 비즈니스 로직을 완전히 분리했습니다

공고 관리 — 채용 공고 등록·수정·삭제, 조건별 검색을 구현했습니다

관리자 통계 — 인기 공고 Top 5, 지역·직종별 통계를 대시보드로 구성했습니다

SQL 인젝션 방지 — PreparedStatement로 모든 파라미터를 바인딩했습니다

개발 중 겪은 문제

문제

한 파일에 메뉴·SQL·결과 처리를 **전부 섞어서** 작성
공고 수정 시 **회원 코드까지 영향**, 코드 500줄 이상으로 관리 불가
사용자 입력을 SQL에 직접 삽입 → **SQL 인젝션 위험**

해결

DAO / DTO / Service 계층 분리 → 변경 영향 범위 최소화
PreparedStatement로 SQL 파라미터 바인딩 → 인젝션 위험 해소
"역할별 코드 분리" 습관이 React에서 **컴포넌트 분리·커스텀 훅 추출**의 기반

사용자 경험 흐름

회원가입 → 공고 검색 → 지원하기 → 결과 확인 → 관리자 통계

구직자는 회원가입 후 공고를 검색하고 지원합니다. 사업장은 지원자를 확인하고 상태를 변경합니다. 관리자는 인기 공고, 지역별 통계를 대시보드에서 한눈에 확인할 수 있습니다.

Result & Learning

- ✓ DAO/DTO 패턴을 처음 적용했고, 이후 모든 프로젝트에서 계층 분리의 기본 원칙이 되었습니다
- ✓ Oracle DB의 시퀀스, 외래키, 제약 조건을 활용한 관계형 설계를 경험했습니다
- ✓ 복잡한 집계 쿼리를 GROUP BY, 서브쿼리, JOIN으로 최적화하는 역량을 확보했습니다

프로젝트 저장소 보기 →